

Rapport de Sprint 2

Federated Learning appliqué à la classification d'images

Cours : 8INF887 – Apprentissage profond

Étudiant : Vinith Naresh Prabakaran

Date : 10 avril 2026

1 Travail réalisé – Sprint 2

1.1 Étude des distributions de données

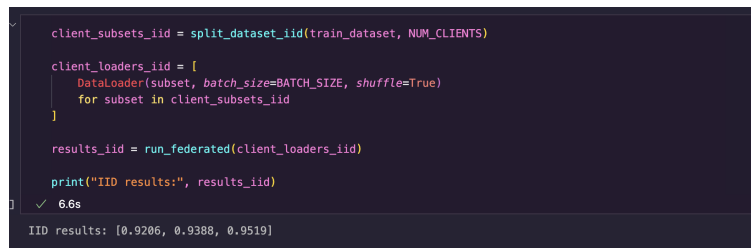
Dans ce sprint, l'objectif principal est d'analyser le comportement du Federated Learning dans des scénarios plus réalistes, en particulier lorsque les données ne sont pas distribuées de manière homogène entre les clients.

Trois types de distribution des données ont été implémentés.

1.1.1 Distribution IID

Dans le cas IID, les données sont réparties de manière uniforme entre les clients.

Cette répartition est réalisée à l'aide de la fonction `random_split` de PyTorch, qui divise le dataset en sous-ensembles de taille équivalente. Chaque client reçoit ainsi une portion du dataset contenant une distribution similaire des classes.



```
client_subsets_iid = split_dataset_iid(train_dataset, NUM_CLIENTS)

client_loaders_iid = [
    DataLoader(subset, batch_size=BATCH_SIZE, shuffle=True)
    for subset in client_subsets_iid
]

results_iid = run_federated(client_loaders_iid)

print("IID results:", results_iid)
```

✓ 6.6s

IID results: [0.9206, 0.9388, 0.9519]

FIGURE 1 – Répartition IID des données entre les clients

1.1.2 Distribution Non-IID par classes

Dans ce scénario, chaque client reçoit uniquement un sous-ensemble de classes.

Concrètement, les labels du dataset sont extraits puis divisés en groupes de classes distincts. Chaque client est ensuite associé à un groupe de classes et ne reçoit que les échantillons correspondant à ces classes.

Par exemple :

- Client 1 : classes 0 à 2
- Client 2 : classes 3 à 5
- etc.

Cette approche simule une situation réaliste où les clients possèdent des données fortement spécialisées.

```

def split_noniid_classes(dataset, num_clients=5):
    targets = np.array(dataset.targets)
    class_splits = np.array_split(np.arange(10), num_clients)

    subsets = []
    for classes in class_splits:
        idx = np.where(np.isin(targets, classes))[0]
        subsets.append(Subset(dataset, idx))

    return subsets

client_subsets_noniid = split_noniid_classes(train_dataset, NUM_CLIENTS)

client_loaders_noniid = [
    DataLoader(subset, batch_size=BATCH_SIZE, shuffle=True)
    for subset in client_subsets_noniid
]

results_noniid = run_federated(client_loaders_noniid)

print("Non-IID (classes) results:", results_noniid)

```

✓ 6.7s

Non-IID (classes) results: [0.26, 0.614, 0.6591]

FIGURE 2 – Répartition non-IID par classes

1.1.3 Distribution Non-IID déséquilibrée

Dans ce cas, tous les clients ont accès à l'ensemble des classes, mais en quantités différentes.

La répartition est réalisée en attribuant un nombre variable d'échantillons à chaque client, créant ainsi un déséquilibre dans les données.

Certains clients disposent donc de beaucoup plus de données que d'autres, ce qui reflète des situations réelles où les sources de données sont inégalement réparties.

```

def split_noniid_imbalanced(dataset, num_clients=5):
    total_len = len(dataset)
    proportions = [0.6, 0.2, 0.1, 0.05, 0.05]

    indices = np.random.permutation(total_len)
    splits = (np.array(proportions) * total_len).astype(int)

    subsets = []
    start = 0
    for s in splits:
        subsets.append(Subset(dataset, indices[start:start+s]))
        start += s

    return subsets

client_subsets_imbalanced = split_noniid_imbalanced(train_dataset, NUM_CLIENTS)

client_loaders_imbalanced = [
    DataLoader(subset, batch_size=BATCH_SIZE, shuffle=True)
    for subset in client_subsets_imbalanced
]

results_imbalanced = run_federated(client_loaders_imbalanced)

print("Non-IID (imbalanced) results:", results_imbalanced)

```

✓ 6.6s

Non-IID (imbalanced) results: [0.9346, 0.9603, 0.9668]

FIGURE 3 – Répartition non-IID déséquilibrée

1.1.4 Analyse des résultats

Les performances obtenues pour ces trois scénarios sont présentées dans la figure suivante.

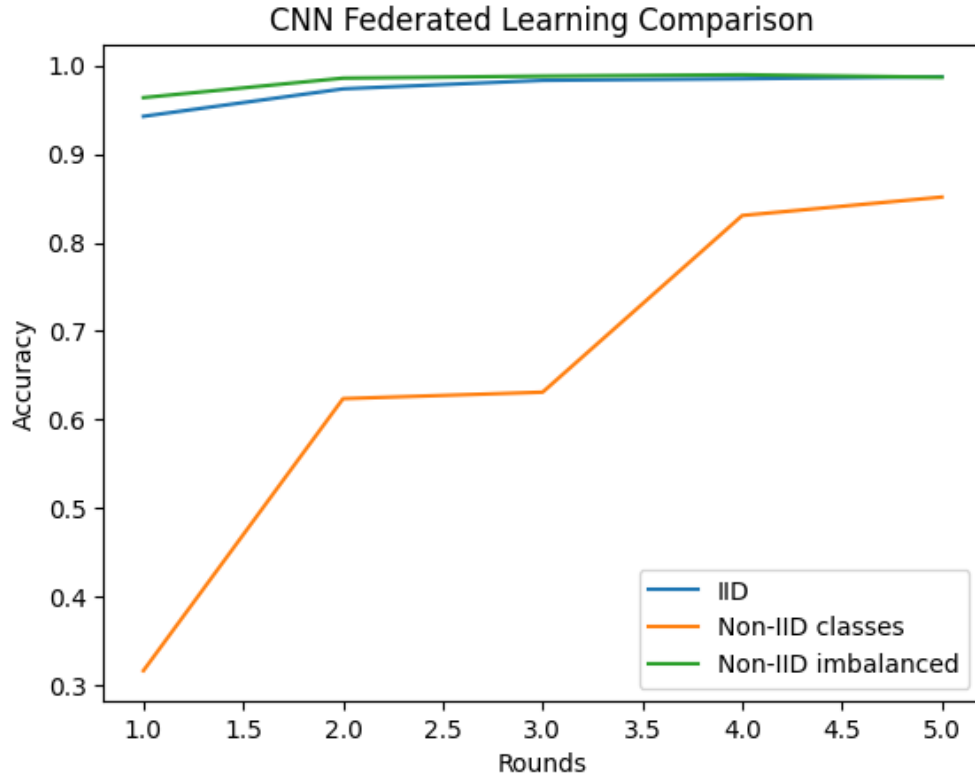


FIGURE 4 – Comparaison des performances selon la distribution des données

1.2 Analyse des performances

Les résultats montrent que :

- Le cas IID atteint des performances élevées et proches du modèle centralisé
- Le cas non-IID par classes dégrade fortement les performances
- Le cas déséquilibré reste performant tant que toutes les classes sont présentes

Cela confirme que la distribution des données est un facteur clé en Federated Learning.

1.3 Impact du nombre de clients

Une étude a été réalisée en faisant varier le nombre de clients.

Lorsque le nombre de clients augmente :

- les données sont plus fragmentées
- l'agrégation devient plus complexe

On observe une légère baisse de performance, ce qui met en évidence les limites de scalabilité du Federated Learning.

```
client_tests = [3, 5, 10]
results_clients = {}

for n_clients in client_tests:
    subsets = split_dataset_iid(train_dataset, n_clients)

    loaders = [
        DataLoader(subset, batch_size=BATCH_SIZE, shuffle=True)
        for subset in subsets
    ]

    res = run_federated(loaders)
    results_clients[n_clients] = res[-1]

print(results_clients)
```

✓ 20.0s

{3: 0.9623, 5: 0.9524, 10: 0.934}

Cette expérience étudie l'effet du nombre de clients sur les performances du modèle.

Une augmentation du nombre de clients entraîne :

- une fragmentation des données
- une complexité accrue de l'agrégation

Cela permet d'analyser la scalabilité du Federated Learning.

FIGURE 5 – Impact du nombre de clients

1.4 Impact du nombre de rounds

L'influence du nombre de rounds a également été étudiée.

Les résultats montrent que :

- une augmentation du nombre de rounds améliore la convergence
- les gains deviennent plus faibles après un certain nombre d'itérations

```
round_tests = [3, 5, 10]
results_rounds = {}

for r in round_tests:
    res = run_federated(client_loaders_iid, rounds=r)
    results_rounds[r] = res[-1]

print(results_rounds)
```

✓ 39.2s

{3: 0.9502, 5: 0.963, 10: 0.9723}

On remarque qu'un plus grand nombre de rounds permet généralement :

- une meilleure convergence
- une amélioration progressive du modèle global

FIGURE 6 – Impact du nombre de rounds

1.5 Amélioration du modèle : CNN

Afin d'améliorer les performances, plusieurs architectures convolutionnelles ont été testées.

1.5.1 CNN V1

Un premier modèle convolutionnel a été introduit afin de capturer la structure spatiale des images, contrairement au MLP.

```
class CNN_V1(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 28 -> 14
        x = self.pool(F.relu(self.conv2(x))) # 14 -> 7

        x = x.view(x.size(0), -1)

        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        return x
```

✓ 0.0s

Ce premier modèle convolutionnel sert de baseline.

Il permet de capturer la structure spatiale des images, contrairement au MLP utilisé précédemment.

Architecture :

- 2 couches convolutionnelles
- pooling
- couches fully connected

FIGURE 7 – Architecture du modèle CNN V1

Les performances obtenues sont les suivantes :

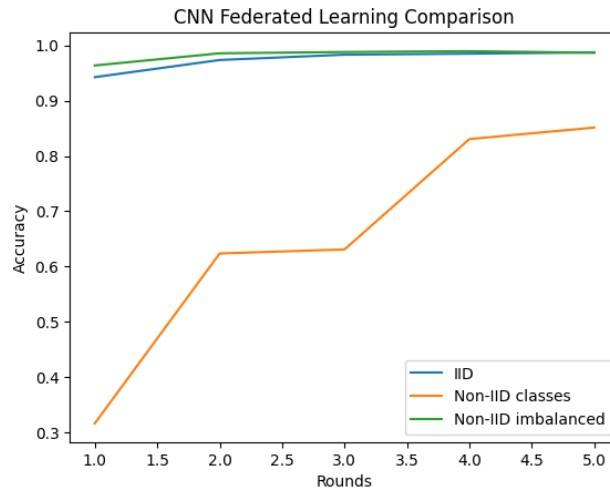


FIGURE 8 – Courbes d'accuracy du CNN V1

```
IID final accuracy: 0.9871
Non-IID classes final accuracy: 0.8513
Non-IID imbalanced final accuracy: 0.9865
```

FIGURE 9 – Résultats numériques du CNN V1

On observe une nette amélioration des performances par rapport au MLP, notamment grâce à une meilleure exploitation des structures spatiales des images.

1.5.2 CNN V2

Une seconde version du modèle a été développée en ajoutant des techniques classiques du deep learning :

- Batch Normalization
- Dropout

```
class CNN_V2(nn.Module):
    def __init__(self):
        super().__init__()

        # Feature extraction
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)

        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

        self.pool = nn.MaxPool2d(2, 2)

        # Classifier
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))

        x = x.view(x.size(0), -1)

        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

        return x
```

✓ 0.0s

Ce modèle introduit :

- Batch Normalization pour stabiliser l'apprentissage
- Dropout pour améliorer la généralisation

FIGURE 10 – Architecture du modèle CNN V2

Les résultats obtenus sont présentés ci-dessous :

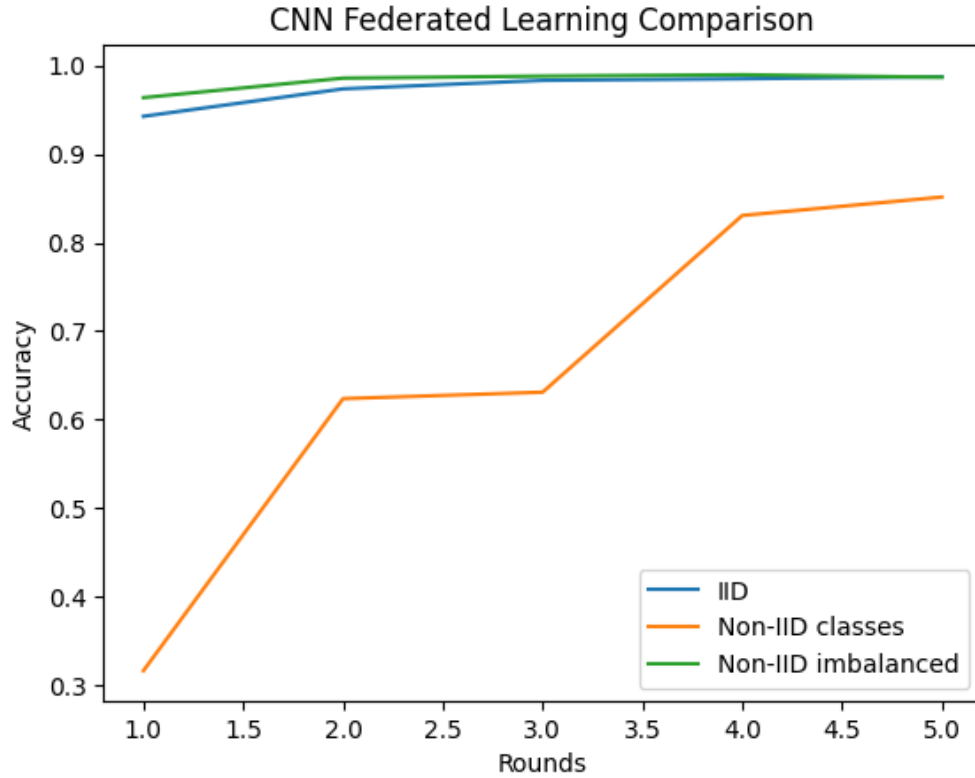


FIGURE 11 – Courbes d’accuracy du CNN V2

```

IID final accuracy: 0.9871
Non-IID classes final accuracy: 0.8513
Non-IID imbalanced final accuracy: 0.9865

```

FIGURE 12 – Résultats numériques du CNN V2

On observe une chute drastique des performances en contexte non-IID, avec une accuracy proche du hasard.

Cela s’explique par l’utilisation de Batch Normalization, qui dépend de statistiques locales incompatibles avec le Federated Learning.

1.5.3 CNN V3

Une troisième version a été proposée afin de corriger les limitations observées précédemment.

Les modifications apportées sont :

- suppression de Batch Normalization
- conservation du Dropout


```

class CNN_V3(nn.Module):
    def __init__(self):
        super().__init__()

        # Feature extraction
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        # Classifier
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout = nn.Dropout(0.4)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 28 → 14
        x = self.pool(F.relu(self.conv2(x))) # 14 → 7

        x = x.view(x.size(0), -1)

        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

        return x

```

✓ 0.0s

Ce modèle est conçu spécifiquement pour le Federated Learning :

- Suppression de BatchNorm
- Utilisation de Dropout uniquement

Objectif : améliorer la robustesse en contexte non-IID.

FIGURE 13 – Architecture du modèle CNN V3

Les résultats obtenus sont les suivants :

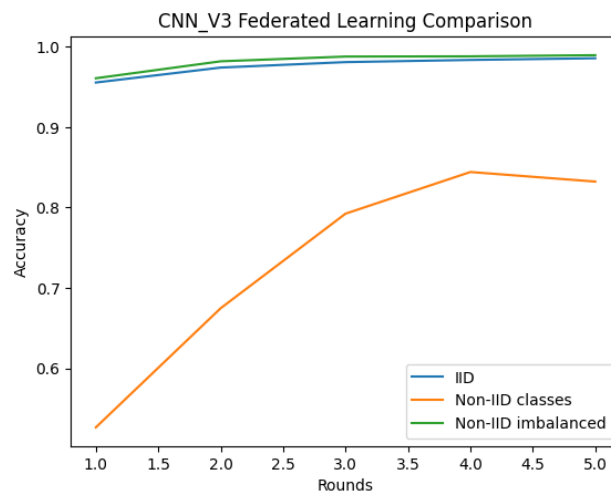


FIGURE 14 – Courbes d'accuracy du CNN V3

```
IID final accuracy: 0.9857
Non-IID classes final accuracy: 0.8323
Non-IID imbalanced final accuracy: 0.9895
```

FIGURE 15 – Résultats numériques du CNN V3

On observe que les performances sont fortement améliorées en contexte non-IID par rapport au CNN V2.

Le modèle atteint des performances proches du cas IID, tout en restant stable.

1.5.4 Comparaison globale

Modèle	IID	Non-IID (classes)	Non-IID (imbalanced)
MLP	~0.95	0.7328	0.9720
CNN V1	~0.987	~0.85	~0.986
CNN V2	~0.985	~0.11	~0.988
CNN V3	~0.985	~0.83	~0.989

TABLE 1 – Comparaison des performances des modèles

Cette comparaison met en évidence que :

- CNN V1 améliore fortement les performances par rapport au MLP
- CNN V2 échoue en contexte non-IID
- CNN V3 offre le meilleur compromis entre performance et robustesse

1.6 Exploration : Autoencodeur

Une approche alternative a été explorée en utilisant un autoencodeur afin de projeter les données dans un espace latent de dimension réduite.

L'idée derrière cette approche est de capturer des représentations plus générales et plus compactes des données, en filtrant le bruit et les variations locales. Cette piste a été envisagée afin d'améliorer la capacité de généralisation du modèle, en particulier dans un contexte non-IID où les distributions de données varient fortement entre les clients.

1.6.1 Architecture de l'autoencodeur

L'autoencodeur est composé de deux parties :

- un encodeur qui compresse les images (28x28) en un vecteur latent de dimension 32
- un décodeur qui reconstruit les images à partir de cette représentation

```
class AutoEncoder(nn.Module):
    def __init__(self):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 32) # latent space
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(32, 128),
            nn.ReLU(),
            nn.Linear(128, 28*28),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        x_recon = self.decoder(z)
        return z, x_recon
```

FIGURE 16 – Architecture de l'autoencodeur

1.6.2 Entraînement de l'autoencodeur

L'autoencodeur est entraîné de manière centralisée en minimisant l'erreur de reconstruction (MSE).

Cela permet d'apprendre une représentation compacte des données.

1.6.3 Dataset latent

Les images sont ensuite transformées en vecteurs latents via l'encodeur. Chaque donnée devient un vecteur de dimension 32 au lieu d'une image.

```
class LatentDataset(torch.utils.data.Dataset):
    def __init__(self, dataset, encoder):
        self.dataset = dataset
        self.encoder = encoder

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        x, y = self.dataset[idx]
        x = x.unsqueeze(0).to(device)

        with torch.no_grad():
            z = self.encoder(x)

        return z.squeeze(0).cpu(), y
```

✓ 0.0s

Les images sont transformées en vecteurs latents via l'encodeur.
Cela réduit fortement la dimension des données et simplifie le problème, mais peut entraîner une perte d'informations importantes pour distinguer les classes.

FIGURE 17 – Transformation des images en représentations latentes

Cette transformation réduit la dimension des données mais peut entraîner une perte d'informations importantes.

1.6.4 Modèle de classification sur latent

Un modèle simple (MLP) est utilisé pour classifier les représentations latentes.

```
class LatentClassifier(nn.Module):
    def __init__(self):
        super().__init__()

        self.fc1 = nn.Linear(32, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

FIGURE 18 – Modèle de classification sur l'espace latent

1.6.5 Résultats

Les performances obtenues sont les suivantes :

```
IID final accuracy: 0.9857
Non-IID classes final accuracy: 0.3130
Non-IID imbalanced final accuracy: 0.9512
```

FIGURE 19 – Résultats numériques avec autoencodeur

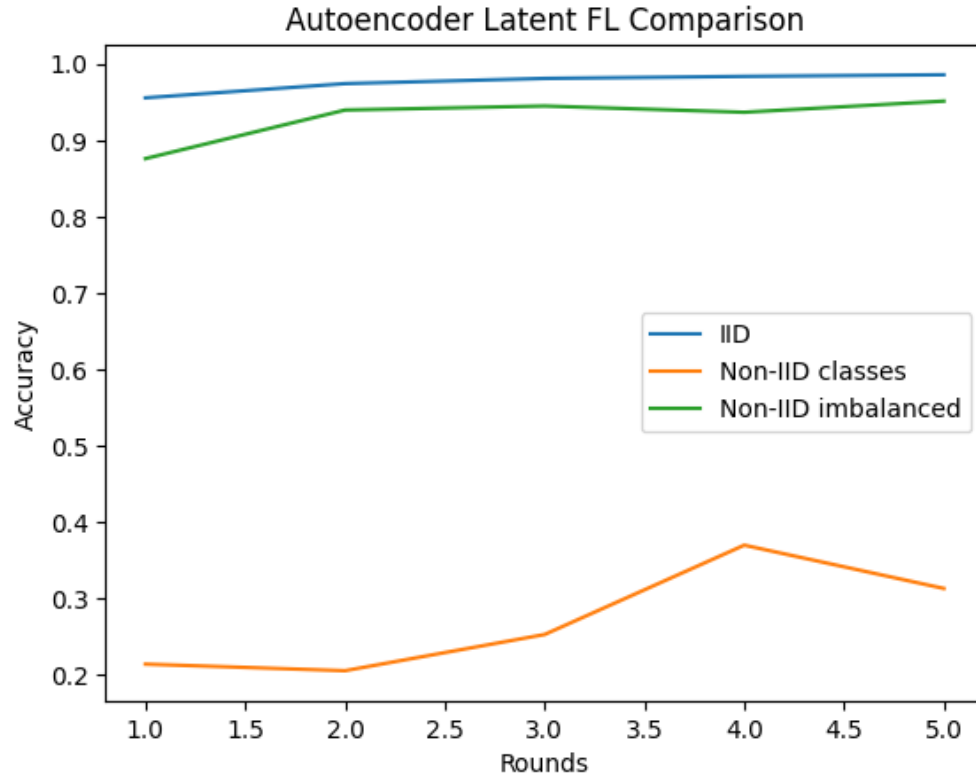


FIGURE 20 – Courbes d’accuracy avec autoencodeur

On observe que :

- les performances restent élevées en contexte IID
- elles chutent fortement en non-IID par classes
- le cas déséquilibré reste performant

1.6.6 Analyse

Ces résultats montrent que la compression des données dans un espace latent entraîne une perte d’informations discriminantes nécessaires à la classification.

Contrairement à l’intuition initiale, l’autoencodeur n’améliore pas la généralisation en contexte non-IID.

1.6.7 Conclusion

L’utilisation d’un autoencodeur constitue une piste intéressante, mais les résultats montrent qu’une représentation latente trop compressée peut dégrader les performances.

Cette expérience met en évidence l’importance de la qualité des représentations dans les systèmes de Federated Learning.

1.7 Analyse globale

Les expériences réalisées permettent de tirer plusieurs conclusions importantes :

- Le Federated Learning fonctionne efficacement en contexte IID
- Le non-IID par classes reste le principal défi
- L’architecture du modèle influence fortement la robustesse
- Certaines techniques classiques (BatchNorm) ne sont pas adaptées
- La qualité de la représentation des données est essentielle

2 Bilan

Ce sprint a permis d’approfondir l’analyse du Federated Learning en étudiant des scénarios plus réalistes.

Les résultats obtenus montrent que la distribution des données a un impact majeur sur les performances, parfois plus important que le choix du modèle lui-même.

Les différentes expérimentations (CNN, autoencodeur) ont permis d’identifier les limites de certaines approches et de proposer des solutions plus adaptées.

Ce travail constitue une étape importante vers une meilleure compréhension des systèmes d’apprentissage distribués et de leurs contraintes en conditions réelles.

Avant la remise finale, plusieurs améliorations sont envisagées afin d’enrichir le projet. Une interface interactive sera développée à l’aide de Streamlit afin de visualiser plus facilement le comportement du modèle et les dynamiques d’apprentissage.

Par ailleurs, une intégration de Substra sera explorée afin de se rapprocher d’un environnement réel de Federated Learning.

Enfin, l’utilisation d’un dataset plus complexe, notamment MIMIC-IV, sera étudiée afin d’évaluer les performances du modèle dans un contexte applicatif plus réaliste, en particulier dans le domaine médical.